

Arama ve Hash'leme

CEN207 Veri Yapıları — Hafta 6

Dr. Öğr. Üyesi Uğur CORUH · 2026-2027 Güz

Bugünün planı (3 saat)

Saat	Konu
1	Arama maliyetleri, sıçramalı Anim 1–2 · enterpolasyon Anim 3–4
2	Üstel Anim 5–6 · Fibonacci Anim 7–8 · hash'leme fikri
3	Bölme Anim 9–10 · zincirleme Anim 11–12 · yoklama Anim 13–18 · yeniden hash Anim 19–20

Öğrenme çıktıları: LO.1 (temel veri yapılarını açıklama) · LO.2 (karmaşıklık analizi) · LO.7 (doğru yapıyı seçme)

Bu haftanın kavramları — nerede

Kavram	Nerede
Sıçramalı, enterpolasyon, üstel, Fibonacci arama	Bölüm 2–5
Hash'leme, bölme yöntemi, asal tablo boyutu	Bölüm 7
Çakışmalar, zincirleme, yük faktörü	Bölüm 8
Açık adresleme, yeniden hash'leme	Bölüm 9–10

Kod örnekleri nasıl çalışır

- Her fikrin eksiksiz bir **C** ve **Java** programı var
- C: `gcc -std=c11 -Wall -Wextra -o /tmp/x file.c && /tmp/x`
- Java: `javac -d /tmp/j File.java && java -cp /tmp/j File`
- Kaynaklar: `code/week-06/c/` ve `code/week-06/java/`
- Her programın beklenen çıktısı hafta notlarında var

Hatırlatma — doğrusal ve ikili arama (Hafta 1)

- **Doğrusal arama (linear search):** her hücreyi tek tek kontrol et — $O(n)$
- **İkili arama (binary search):** sıralı dizi, aralığı ikiye böl — $O(\log n)$
- İkili arama önce dizinin **sıralı** olmasını ister
- Bugün: bu ikisinin arasında ve ötesinde stratejiler

Hatırlatma — diziler ve bağlı listeler (Hafta 2)

- **Dizi (array):** bitişik bellek, indeksle $O(1)$ erişim
- **Bağlı liste (linked list):** düğüm ve işaretçiler, başa $O(1)$ ekleme
- Aşağıdaki arama yapıları doğrudan ikisinin üzerine kurulur
- Bir hash tablosunun kovası (bucket) genelde küçük bir bağlı listedir

Haftanın haritası — genel bakış

Daha hızlı arama (sıralı)	Hash'leme
Sıçramalı, enterpolasyon arama	Doğrudan adresleme, bölme yöntemi
Üstel, Fibonacci arama	Zincirleme, açık adresleme, yeniden hash'leme

1. Arama Problemi ve Maliyetler

Başlangıç sorusu

İkili arama zaten yalnızca $O(\log n)$ alıyor.
Daha da hızlı, ya da tamamen farklı bir şey
aramak için bir neden var mı?

Burada "arama" ne demek

- Bir koleksiyon ve bir **hedef (target)** anahtar verildiğinde, konumunu bul
- Ya da doğru şekilde bildir: anahtar **yok**
- Maliyet olarak **karşılaştırma** (ya da **yoklama/probe**) sayarız
- Aynı doğrulukla daha az karşılaştırma = daha iyi bir arama

Maliyet, soru olarak yeniden

- Doğrusal arama: veri üzerinde varsayım yok, en kötü durum $O(n)$
- İkili arama: **sıralı** veri ister, en kötü durum $O(\log n)$
- Sıralı veri $O(\log n)$ 'den de hızlı aranabilir mi?
- Hiç **karşılaştırma yapmadan** aranabilir mi?

İkili arama ne zaman yetmez

- İkili arama ne olursa olsun her zaman **ortayı** kontrol eder
- Ama gerçek veride ikili aramanın yok saydığı **yapı** olabilir
- Değerler düzgün dağılmışsa: bir formül "ortadan" daha iyi tahmin edebilir
- Bellek dışı veri (disk, teyp): her erişim önemli

Bugünün araç kutusu — beş strateji

Strateji	Neyi kullanır
Sıçramalı arama	Ucuz blok sıçramaları çok karşılaştırmayı geçer
Enterpolasyon araması	Değerler kabaca düzgün dağılmış
Üstel arama	Hedef muhtemelen başa yakın
Fibonacci araması	Bölme/çarpma gerekmez

Mini soru

İkili arama her zaman önce dizinin **orta** elemanına bakar. Hangi durumda daha akıllı bir ilk tahmin daha iyi olabilir?

Yanıt

Değerler kabaca düzgün dağılmışsa.

Orta yalnız "indeks-optimal"dir — değeri hesaba katan bir tahmin (Bölüm 3) hedefe daha yakın düşebilir.

2. Sıçramalı Arama (Jump Search)

Başlangıç sorusu

İkili arama her seferinde **ortaya** sıçrar.
Ya bunun yerine **sabit boyutlu bloklarla**
sıçramak daha basit, ama yine de hızlı olsaydı?

Kısa tarihçe

- Blok tarzı arama, Knuth'un *Sorting and Searching* (1973) kitabında, ikili aramaya bir alternatif olarak geçer
- Kilit tasarım kararı: her blok ne kadar büyük olmalı?
- Yanıt, `block = floor(sqrt(n))`, iki maliyeti dengeler

Benzetme — bir kitapta sayfa atlamak

- Kelimeyi bulmak için tek tek sayfa çevirmek: çok yavaş
- İkili arama: kalan sayfaları hep ikiye böl
- Sıçramalı arama: önce **tüm bölümleri** atla, sonra bir sayfa
- Kelimeyi geçtiğinde büyük sıçramaları durdur

Sıçramalı arama fikri

- Tıpkı ikili arama gibi **sıralı** bir dizi gerektirir
- `block = floor(sqrt(n))` : sıçrama mesafesi
- Bir sınır `>= target` olana kadar bir blok ileri sıçra
- Sonra o tek bloğu soldan sağa **doğrusal** tara

Neden $\text{sqrt}(n)$?

- n/block sıçrama, ardından en çok block doğrusal adım
- Toplam maliyet, en kötü durumda: $n/\text{block} + \text{block}$
- Bu toplam $\text{block} = \text{sqrt}(n)$ iken en küçük olur
- Sonuç: $O(\text{sqrt}(n))$ — $O(\log n)$ ile $O(n)$ arasında

Sıçramalı arama, adım adım

target = 42

$\text{block} = \text{floor}(\text{sqrt}(16)) = 4$

karşılaştırma: 6



Sonuç: $\text{arr}[10] = 42$, 6 karşılaştırmada bulundu (3 sıçrama + doğrusal tarama). $O(\text{sqrt}(n))$.

Uç durum — hedef aralıkta ama dizide yok

target = 45

$\text{block} = \text{floor}(\text{sqrt}(16)) = 4$

karşılaştırma: 7



Bulunamadı: 45 dizide yok. Toplam 7 karşılaştırma — doğrusal aramanın $n = 16$ karşılaştırmasından çok daha az: $O(\text{sqrt}(n))$.

Kod — doğru bloğu bulmak

```
int block = (int) sqrt((double) n);
if (block < 1) block = 1;
int prev = 0, step = block, comp = 0;
while (step < n) {
    comp++;
    if (arr[step - 1] >= target) break;
    prev = step;
    step += block;
}
```


Kod — blok içinde doğrusal tarama

```
if (step > n) step = n;
for (int i = prev; i < step; i++) {
    comp++;
    if (arr[i] == target) return i;
    if (arr[i] > target) break;
}
return -1;
```

Karmaşıklık

- Blok bulma aşaması: en çok n/block sıçrama
- Doğrusal tarama aşaması: en çok block karşılaştırma
- $\text{block} = \text{sqrt}(n)$ ile: toplam $O(\text{sqrt}(n))$
- İkili aramanın $O(\log n)$ 'inden kötü, ama kod daha basit

Sık yapılan hatalar

- Dizinin önce **sıralı** olması gerektiğini unutmak
- `sqrt(n)` yerine sabit bir blok boyutu kullanmak
- Durması gerekirken blok sınırını aşarak taramaya devam etmek

Mini soru

Bir dizide $n = 100$ eleman var. Sıçramalı arama hangi blok boyutunu kullanır, ve en kötü durumda kaç karşılaştırma gerekir?

Yanıt

block = 10 ($\text{sqrt}(100)$). En kötü durum:
yaklaşık **10** sıçrama artı **10** doğrusal adım —
~20 karşılaştırma, doğrusal aramanın **100** 'ünden çok az.

3. Enterpolasyon Araması (Interpolation Search)

Başlangıç sorusu

Bir telefon rehberi sıralı **ve** kabaca düzgün dağılmış. "Smith"i aramaya gerçekten tam orta sayfadan mı başlardınız?

Kısa tarihçe

- Fikir 1957'de, sıralı dosyalarda arama üzerine erken çalışmalarda görülür (W. W. Peterson)
- Sonra **enterpolasyon araması** olarak biçimlenip incelenir
- **Düzgün** veride en iyi durum: $O(\log \log n)$ 'e yakın

Benzetme — bir telefon rehberi

- "Smith" aranıyor: ortaya değil, arkaya yakın bir yere sıçra
- "Baker" aranıyor: öne yakın bir yere sıçra
- Sıçrama tahmini **değeri** kullanır, yalnız konumu değil
- İkili arama yalnız konumu kullanır — değerleri yok sayar

Enterpolasyon araması fikri

- Tıpkı ikili arama gibi **sıralı** bir dizi gerektirir
- `target` 'ın konumunu uçlar arasında **orantılı** tahmin et
- $$\text{pos} = \text{lo} + (\text{target} - \text{arr}[\text{lo}]) * (\text{hi} - \text{lo}) / (\text{arr}[\text{hi}] - \text{arr}[\text{lo}])$$
- `lo` / `hi` 'yi `pos` 'a doğru daralt, tıpkı ikili arama gibi

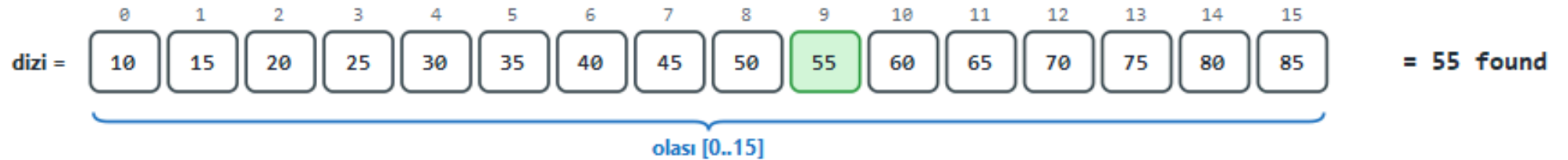
Enterpolasyon araması, adım adım

target = 55

ikili arama burada ~4 karşılaştırma yapardı

yoklama: 1

$$\text{pos} = 0 + \text{floor}((55 - 10) * (15 - 0) / (85 - 10)) = 9$$



Sonuç: $\text{arr}[9] = 55$, yalnız 1 yoklamada bulundu. Düzgün dağılmış veride enterpolasyon araması ikili aramadan (~4 karşılaştırma) çok daha az adımda sonuca ulaşır.

Uç durum — sifıra bölmeye karşı koruma

target = 42

ikili arama burada ~4 karşılaştırma yapardı

yoklama: 1

`arr[hi] == arr[lo] (42)` – bölme yapılamaz, korumaya gir: bu aralık tek bir değerden oluşuyor.



Sonuç: `arr[0] = 42`, yalnız 1 yoklamada bulundu. Düzgün dağılmış veride enterpolasyon araması ikili aramadan (~4 karşılaştırma) çok daha az adımda sonuca ulaşır.

Kod — konum formülü

```
while (lo <= hi && target >= arr[lo]
      && target <= arr[hi]) {
    if (arr[hi] == arr[lo]) {
        /* ... koruma: sıfıra bölme ... */
        return lo;
    }
    int pos = lo + (int) ((double)
        (target - arr[lo]) * (hi - lo)
        / (arr[hi] - arr[lo]));
```

Kod — aralığı daraltmak

```
if (arr[pos] == target) return pos;  
if (arr[pos] < target) lo = pos + 1;  
else hi = pos - 1;
```

Karmaşıklık

- Düzgün veri: ortalamada $O(\log \log n)$ 'e yakın
- Çarpık veri (tek büyük aykırı değer): $O(n)$ 'e doğru bozulur
- En kötü durum ikili aramanın $O(\log n)$ 'inden asla iyi değil
- Formülün gücü tamamen verinin şekline bağlı

Sık yapılan hatalar

- `arr[hi] == arr[lo]` bölme korumasını unutmak
- Enterpolasyon aramasını **çarpık** (düzgün olmayan) veride kullanmak
- $O(\log \log n)$ 'i bir en-kötü-durum garantisi sanmak — değil

Mini soru

Bir dizinin değerleri aşırı **çarpık** —
çoğu küçük, sonda tek bir dev aykırı değer.
Enterpolasyon araması yine de iyi bir seçim mi?

Yanıt

Hayır, illa değil. Çarpık veri, enterpolasyon aramasının maliyetini $O(n)$ 'e itebilir — sade ikili aramanın $O(\log n)$ 'i daha güvenli seçim olur.

4. Üstel Arama (Exponential Search)

Başlangıç sorusu

Devasa, **sıralı** bir veri akışında arama yapıyorsunuz ve uzunluğunu bilmiyorsunuz. İkili arama nereden başlayabilir ki?

Kısa tarihçe

- **1976** — Jon Bentley ve Andrew Yao, "An almost optimal algorithm for unbounded searching"ı yayımlar
- **Sınırsız (unbounded)** ya da **galloping** arama da denir
- Bugün bazı birleştirme ve küme kesişim algoritmalarında kullanılır

Benzetme — sınırsız bir listede arama

- Bilinen bir son yok mu? Sonda büyüyen bir **tahmin** kullan
- İndeks 1'e, sonra 2'ye, 4'e, 8'e, 16'ya bak, ...
- Açıkça çok ileri gidince ikiye katlamayı durdur
- Şimdi sıradan, **sınırlı** bir ikili arama işi bitirir

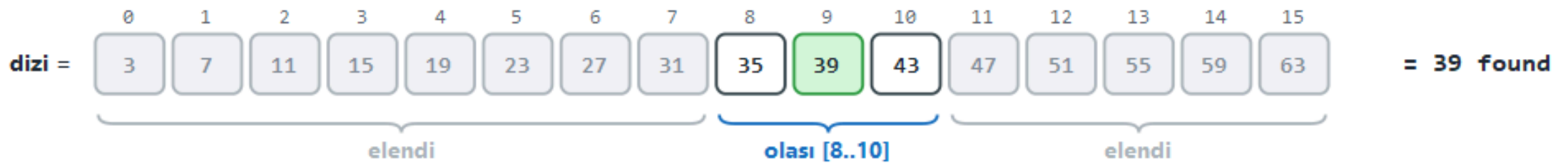
Üstel arama fikri

- Önce `arr[0]` 'a bak — hedef tam orada olabilir
- `bound` 'u büyüt: 1, 2, 4, 8, ... `arr[bound] >= target` olana dek
- `[bound/2, bound]` içinde sıradan **ikili arama** çalıştır
- Şimdiye kadarki her strateji gibi **sıralı** bir dizi gerektirir

Üstel arama, adım adım

target = 39

karşılaştırma: 7



Sonuç: $arr[9] = 39$, 7 karşılaştırmada bulundu. Hedef başa yakınsa sınır bulma çok ucuzdur.

Uç durum — hedef son değerden de büyük

target = 999

karşılaştırma: 9



Bulunamadı: 999 dizide yok. Toplam 9 karşılaştırma: sınır bulma $O(\log \text{index})$, sonrası ikili arama $O(\log(\text{bound}))$.

Kod — sınırı büyütmek

```
int comp = 1;
if (arr[0] == target) return 0;
int bound = 1;
while (bound < n) {
    comp++;
    if (arr[bound] >= target) break;
    bound *= 2;
}
```

Kod — sınır içinde ikili arama

```
int lo = bound / 2;
int hi = (bound < n) ? bound : n - 1;
while (lo <= hi) {
    int mid = lo + (hi - lo) / 2;
    comp++;
    if (arr[mid] == target) return mid;
    if (arr[mid] < target) lo = mid + 1;
    else hi = mid - 1;
}
```

Karmaşıklık

- Sınır bulma aşaması: $O(\log \text{index})$, `index` hedefin gerçekte bulunduğu yer
- İkili arama aşaması: $O(\log(\text{bound}))$, yani o da $O(\log \text{index})$
- Toplam: $O(\log \text{index})$ — hedef başa yakınken $O(\log n)$ 'den çok daha iyi

Sık yapılan hatalar

- Sona yakinken `bound` 'u `n - 1` 'e sabitlemeyi unutmak
- İkili aramayı `[bound/2, bound]` yerine `[0, bound]` 'dan yeniden başlatmak — ikiye katlama aşamasının işini boşa harcar
- Bunun bilinen bir `n` gerektirdiğini varsaymak — açıkça gerektirmez

Mini soru

Bir dizide bir milyon eleman var, ama hedef indeks 3'te. Üstel arama yaklaşık kaç karşılaştırma gerektirir?

Yanıt

Yalnız birkaç tane — dizi bir milyon eleman tutsa da, toplam yaklaşık $\log_2(3) + 1 \approx 3$ karşılaştırma.

5. Fibonacci Araması (Fibonacci Search)

Başlangıç sorusu

Şimdiye kadarki her strateji bölüyor:

n/block , $(hi-lo)/2$, $\text{bound}/2$. Ya

programınızı çalıştıran donanım ucuza bölemiyorsa?

Kısa tarihçe

- **1953** — Jack Kiefer'ın Fibonacci arama **teknîği**, önce dizi için değil, optimizasyon için geliştirildi
- Kısa süre sonra dizi aramasına uyarlandı
- Yalnız **toplama ve çıkarma** kullanır — **/** yok, ***** yok

Benzetme — cetvel olarak Fibonacci sayıları

- Fibonacci sayıları: 1, 1, 2, 3, 5, 8, 13, 21, ...
- Her biri kendinden önceki ikisinin toplamı
- n 'e $\geq$ en küçük Fibonacci sayısını "cetvel" olarak seç
- Aralığı bölerek değil, bu cetveli kullanarak böl

Fibonacci araması fikri

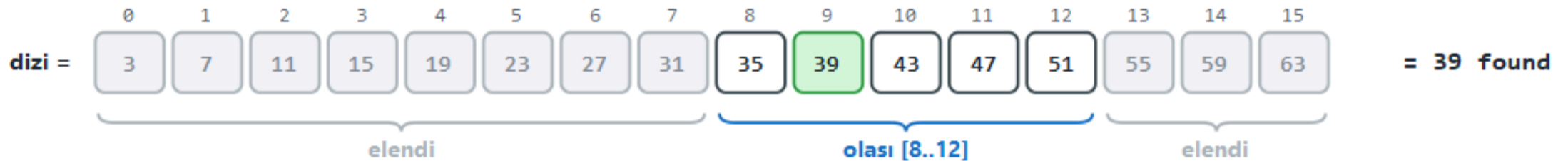
- Tıpkı ikili arama gibi **sıralı** bir dizi gerektirir
- Bir Fibonacci adımı arayla `(fib, fib1, fib2)` üçlüsünü izle
- `offset + fib2` 'de yokla; karşılaştı, sonra üçlüyü küçült
- `< target` : bir adım küçült; `> target` : iki adım küçült

Fibonacci araması, adım adım

target = 39

fib=5, fib1=3, fib2=2, offset=7

karşılaştırma: 3



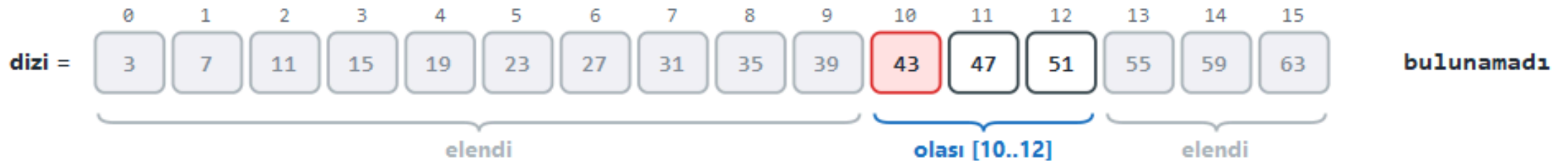
Sonuç: $arr[9] = 39$, 3 karşılaştırmada bulundu. Fibonacci araması yalnız toplama/çıkarma kullanır — çarpma ya da bölme yok.

Uç durum — hedef aralıkta ama dizide yok

target = 40

fib=1, fib1=1, fib2=0, offset=9

karşılaştırma: 5



Bulunamadı: 40 dizide yok. Toplam 5 karşılaştırma. Fibonacci araması yalnız toplama/çıkarma kullanır — çarpma ya da bölme yok.

Kod — Fibonacci üçlüsü ve yoklama

```
int fib2=0, fib1=1, fib=fib1+fib2;
while (fib < n) {
    fib2=fib1; fib1=fib; fib=fib1+fib2;
}
int offset = -1;
while (fib > 1) {
    int i = (offset+fib2 < n-1)
        ? offset+fib2 : n-1;
```

Kod — üçlüyü küçültmek

```
if (arr[i] < target) {  
    fib=fib1; fib1=fib2; fib2=fib-fib1;  
    offset = i;  
} else if (arr[i] > target) {  
    fib=fib2; fib1=fib1-fib2; fib2=fib-fib1;  
} else {  
    return i;  
}
```


Karmaşıklık

- İkili aramayla aynı mertebe: $O(\log n)$
- Pratikte ortalamada biraz daha fazla karşılaştırma
- Gerçek avantajı: **bölme ya da çarpma yok**
- Hızlı bölmesi olmayan donanımda tarihsel olarak değerli

Sık yapılan hatalar

- Sonda kalan tek elemanın kontrolünü unutmak
- Fibonacci üçlüsünü ana döngünün **içinde** hesaplamak
- İkili aramanın $O(\log n)$ 'ini geçtiğini varsaymak — geçmez

Mini soru

Fibonacci araması neden 1950'ler ve 1960'larda, bugünün donanımından daha çok önem taşımış olabilir?

Yanıt

Bölme, erken donanımda pahalıydı ya da mevcut değildi. Fibonacci aramasının saf toplama/çıkarması bu maliyeti tamamen ortadan kaldırıyordu.

6. Beş Arama Stratejisini Karşılaştırmak

Karşılaştırma — her biri ne ister

Strateji	İster	En iyi durum
Sıçramalı arama	Sıralı dizi	$O(\sqrt{n})$
Enterpolasyon araması	Sıralı, düzgün	$O(\log \log n)$
Üstel arama	Sıralı dizi	$O(\log \text{ index})$

Karşılaştırma — Fibonacci ve hatırlatma

Strateji	İster	En iyi durum
Fibonacci araması	Sıralı dizi	$O(\log n)$
İkili arama (Hafta 1)	Sıralı dizi	$O(\log n)$
Doğrusal arama (Hafta 1)	Hiçbir şey	$O(n)$

Hangisi ne zaman kullanılır

- Veri kabaca düzgün mü? **Enterpolasyon** araması
- Hedef genelde başa yakın mı? **Üstel** arama
- Basit kod, bölme yok mu? **Sıçramalı** ya da **Fibonacci**
- Özel bir yapı yok mu? Sade **ikili arama** yeterli

Mini soru

Sıralı bir günlük dosyası sürekli büyüyor, uzunluğu **n** önceden hiç bilinmiyor. Bugünün hangi stratejisi en uygun, neden?

Yanıt

Üstel arama. Yalnız aşana kadar bir sınırı büyütür — ikili aramadan farklı olarak n 'in önceden bilinmesine hiç gerek duymaz.

7. Hash'leme: Doğrudan Adreslemeden Hash Fonksiyonlarına

Başlangıç sorusu

Şimdiye kadarki her arama değerleri tekrar tekrar **karşılaştırıyor**. Bunun yerine bir anahtarın konumunu tek adımda **hesaplayabilir** miyiz?

Kısa tarihçe

- **1953** — IBM'de Hans Peter Luhn, kayıtları doğrudan yerleştirmek için hesaplanmış bir fonksiyon önerir
- Bu iç yazı, hash'lemenin kökeni olarak geniş çapta kabul edilir
- Fikir: bir **anahtarı**, bir formülle **tablo indeksine** çevir

Benzetme — doğrudan adresleme

- Anahtarlar küçük tamsayılar, $0..m-1$ mi? Anahtarın kendisini dizi indeksi olarak kullan — $O(1)$, hesaplama yok
- `table[key] = value` — doğrudan adresleme, hiç hash yok
- Sorun: gerçek anahtarlar $0..m-1$ 'e nadiren düzgün sığar
- Öğrenci numaraları, telefon numaraları: çok fazla olası değer

Hash'leme fikri

- Bir **hash fonksiyonu** $h(\text{key})$, herhangi bir anahtarı $0 \dots m-1$ 'e eşler
- m = tablo boyutu; h hızlı olmalı, idealde $O(1)$
- İki farklı anahtar aynı indekse düşerse: bir **çakışma (collision)**
- Bölüm 8–10, çakışmaları ele alan mekanizmayı kurar

Bölme yöntemi

- En basit yaygın hash fonksiyonu: $h(k) = k \bmod m$
- Herhangi bir tamsayı anahtarı $[0, m-1]$ 'e eşler — her zaman, $O(1)$
- **m'nin seçimi**, anahtarların ne kadar iyi dağıldığını değiştirir
- Dikkatsiz bir m , birçok anahtarı bilerek çakıştırabilir

Bölme yöntemi, adım adım

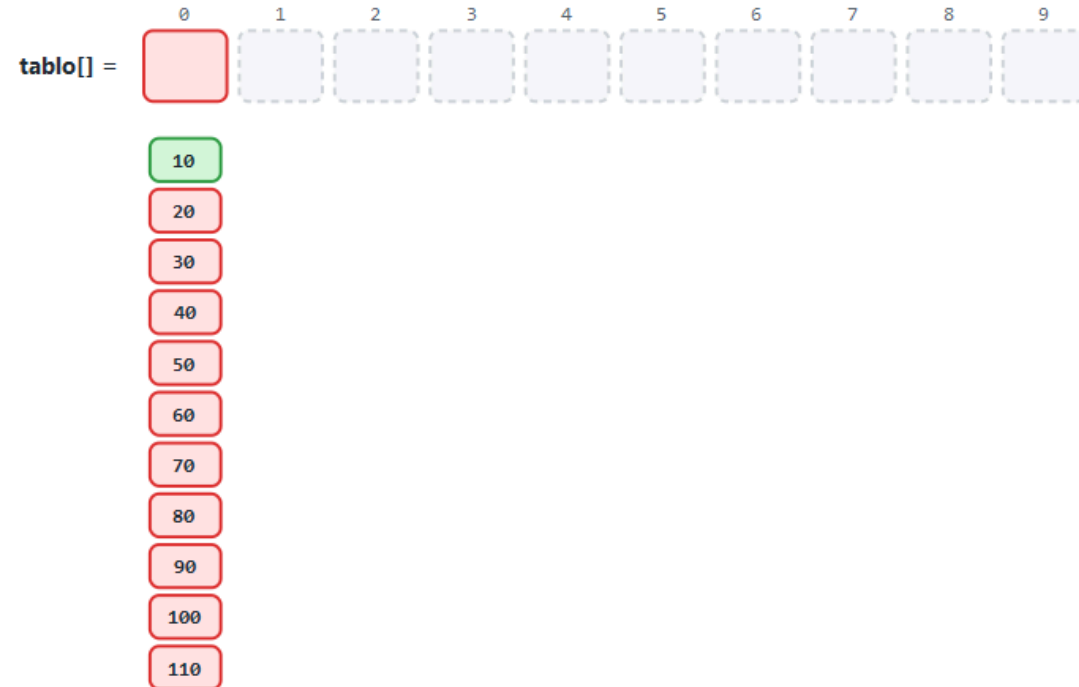
$m = 11$



Bitti: 12 anahtar, 4 çakışma, 8/11 hücre kullanıldı. m 'in seçimi kritik: anahtarların örüntüsüyle ortak çarpanı olmayan (genelde asal) bir m , kümelenmeyi azaltır.

Uç durum — m 10'un kuvveti: felaket

m = 10



$$h(110) = 110 \bmod 10 = 0$$

Bitti: 11 anahtar, 10 çakışma, 1/10 hücre kullanıldı. m'in seçimi kritik: anahtarların örüntüsüyle ortak çarpanı olmayan (genelde asal) bir m, kümelenmeyi azaltır.

Neden m asal olmalı?

- m ile **ortak çarpanı** olan anahtarlar kötü çakışır
- m 10'un kuvveti, anahtarlar 10'un katı: toplam çöküş
- **Asal** bir m , çoğu anahtar örüntüsüyle çarpan paylaşmaz
- Kural: m 'yi asal seçin, 2'nin ya da 10'un kuvvetlerinden kaçının

Kod — bölme hash fonksiyonu

```
/* + m) % m, C'de negatif anahtarlara karşı korur */  
int hash_division(int key, int m) {  
    return ((key % m) + m) % m;  
}
```

Karmaşıklık

- $h(k) = k \bmod m$: tek bir bölme — her zaman **$O(1)$**
- Hash'lemeyi çekici kılan tam olarak bu $O(1)$ maliyeti
- Bir hash tablosunun gerçek maliyeti **çakışmalarda** yaşar
- Bölüm 8–10 tamamen bu maliyeti yönetmekle ilgili

Sık yapılan hatalar

- Kolaylık olsun diye `m`'yi 2 ya da 10'un kuvveti seçmek
- C ya da Java gibi dillerde negatif-anahtar korumasını unutmak
- Çok fazla çakışma olsa bile $O(1)$ 'in her zaman geçerli olduğunu sanmak

Mini soru

$m = 8$ ve bir veri kümesindeki her anahtar **çift** sayı. Tablonun hücrelerinin ne kadarı hiç kullanılabilir?

Yanıt

En çok yarısı. Çift bir anahtar mod 8 yalnız çift bir indekse (0, 2, 4, 6) düşebilir — tek indeksli hücrelere hiç ulaşamaz.

8. Çakışmalar ve Ayrık Zincirleme (Separate Chaining)

Başlangıç sorusu

İki farklı anahtar aynı indekse hash'leniyor.
Tabloda orada yalnız bir hücre var. İkinci
anahtar nereye gidiyor?

Benzetme — paylaşılan bir posta kutusu

- İki öğrenci rastlantıyla aynı posta kutusu numarasını paylaşıyor
- Çözüm: o tek posta kutusuna **küçük bir liste** as
- O bir hücreyi istediğiniz kadar anahtar, art arda paylaşabilir
- Bu **ayrık zincirleme**: her kovaya (bucket) bir bağlı liste

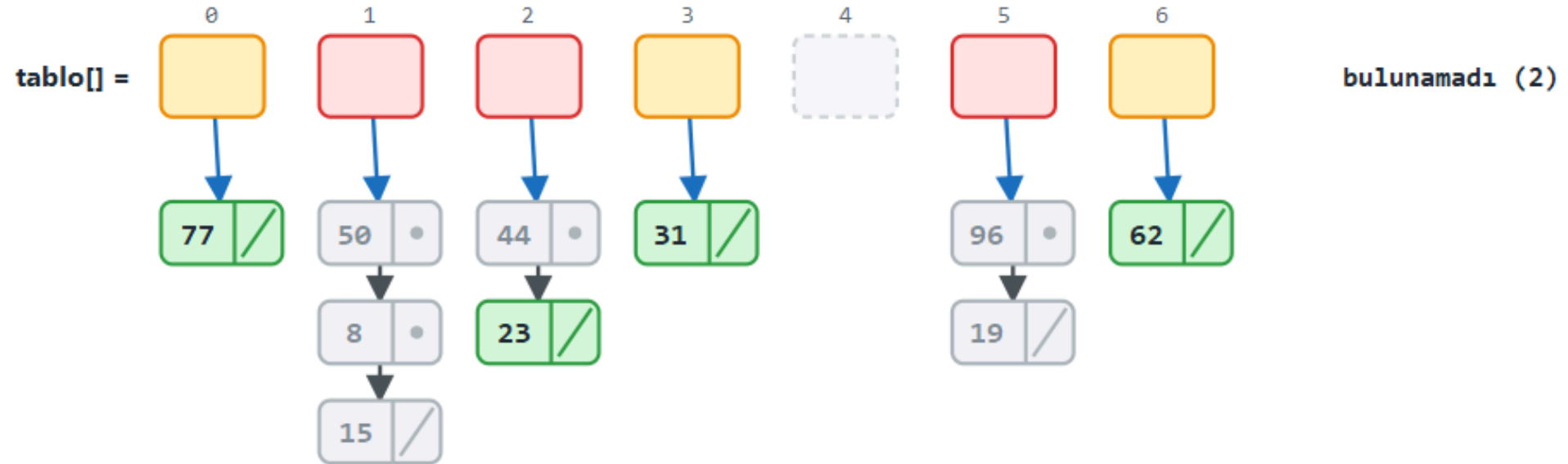
Zincirleme fikri

- Tablo hücresi `i`, bir bağlı listenin **başını** tutar
- Ekleme: yeni düğüm listenin **başı** olur — $O(1)$
- Arama: listeyi dolaş, her anahtarı karşılaştır — $O(\text{zincir uzunluğu})$
- Boş zincir (`NULL`): anahtar kesinlikle yok

Ayrık zincirleme, adım adım

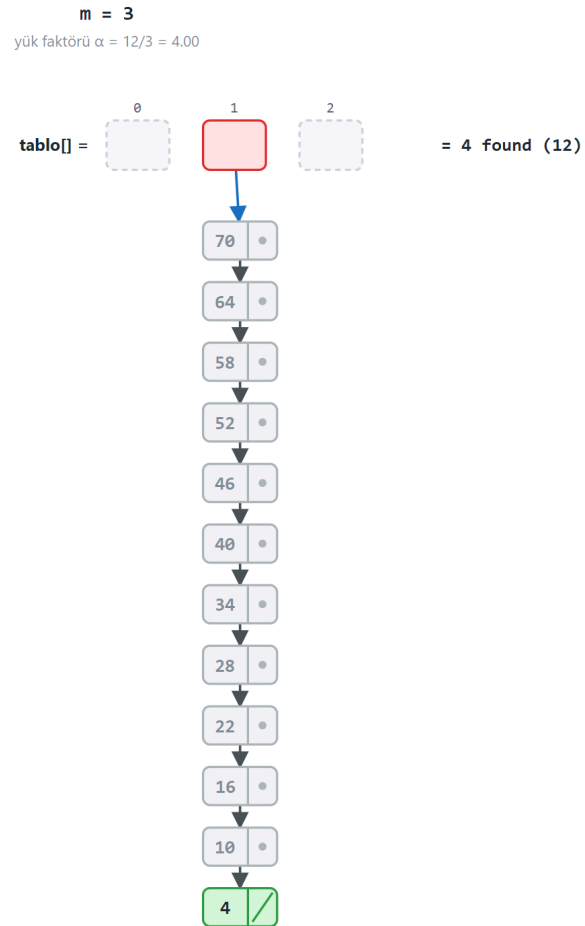
$$m = 7$$

$$\text{yük faktörü } \alpha = 10/7 = 1.43$$



Bitti: 10 anahtar, $\alpha = 1.43$. 4 arama: 2 isabet, 2 kaçırma. Arama maliyeti zincir uzunluğuna bağlı, ortalama $O(1 + \alpha)$.

Uç durum — ağır yüklü bir tablo



Bitti: 12 anahtar, $\alpha = 4.00$. 3 arama: 2 isabet, 1 kaçırma. Arama maliyeti zincir uzunluğuna bağlı, ortalama $O(1 + \alpha)$.

Kod — ekleme (zincirin başı)

```
void insert(int key) {  
    int idx = key % M;  
    Node *n = malloc(sizeof(Node));  
    n->key = key;  
    n->next = table[idx]; /* yeni baş */  
    table[idx] = n;  
}
```

Kod — arama (zinciri dolaşmak)

```
bool search(int key) {  
    int idx = key % M;  
    for (Node *c = table[idx]; c;  
         c = c->next) {  
        if (c->key == key) return true;  
    }  
    return false;  
}
```


Yük faktörü (load factor)

- $\alpha = n / m$ — saklanan anahtarlar, tablo boyutuna bölünmüş
- $\alpha = 0,5$: zincirler ortalama 0,5 uzunlukta — neredeyse $O(1)$
- $\alpha = 4$: zincirler ortalama 4 uzunlukta — gerçek maliyet, $O(1)$ değil
- Bir hash tablosunun **hızı** doğrudan α 'ya bağlı

Karmaşıklık

- Ekleme: her zaman $O(1)$ — doğrudan zincirin başına
- Arama (ortalama): $O(1 + \alpha)$ — bir hash, artı zincir
- Arama (en kötü durum): $O(n)$ — her anahtar tek bir zincirde
- α 'yı küçük tutmak, zincirlemeyi pratikte hızlı tutan şey

Sık yapılan hatalar

- Tabloyu hiç büyütmeden α 'nın sınırsız büyümesine izin vermek
- Boş bir zincirin (NULL) "kesinlikle yok" demek olduğunu unutmak
- "Çakışma"yı "hata" ile karıştırmak — çakışmalar beklenir

Mini soru

Bir tabloda $m = 5$ kova ve zincirlemeyle düzgün dağılmış $n = 20$ anahtar var. Yük faktörü ve ortalama zincir uzunluğu nedir?

Yanıt

$\alpha = 20 / 5 = 4$. Ortalamada, her zincir yaklaşık 4 anahtar tutar — arama $O(1)$ değil, kabaca 4 karşılaştırmaya mal olur.

9. Açık Adresleme (Open Addressing)

Başlangıç sorusu

Ayrık zincirleme liste düğümleri için ekstra bellek ister. Her anahtar bunun yerine **doğrudan tablonun hücrelerinde** yaşayabilir mi?

Açık adresleme fikri

- Liste yok: her anahtar doğrudan tablo dizisinde yaşar
- $h(key)$ 'de çakışma mı? Bir dizi başka hücreyi **yokla (probe)**
- İlk **boş** hücrede dur — anahtar oraya yerleşir
- Silme bir **mezar taşı (tombstone)** işareti ister, gerçek boş hücre değil
- Aşağıda üç yoklama kuralı: doğrusal, karesel, çift hash

9a. Doğrusal Yoklama (Linear Probing)

Doğrusal yoklama fikri

- $h(\text{key})$ 'de çakışma mı? $h(\text{key})+1$, $+2$, $+3$, ... dene
- İndeks sonu geçince $\text{mod } m$ ile başa sar
- Boş **ya da** silinmiş ilk hücrede dur
- Kodlaması basit, ama anahtarlar **öbekler** halinde yığılır

Doğrusal yoklama, adım adım

m = 11



boş hücre – dur

h = 1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 10 → 0

Bitti: 10/10 ekleme yerleşti. Doğrusal yoklama basittir ama birincil kümelenmeye yol açar: dolu bloklar büyüdükçe yeni çakışmalar daha da büyütür.

Uç durum — silme sonrası arama

m = 11



Bitti: 10/10 ekleme yerleşti. Doğrusal yoklama basittir ama birincil kümelenmeye yol açar: dolu bloklar büyüdükçe yeni çakışmalar daha da büyütür.

Uç durum — tablo doluyor

$m = 8$



tablo dolu!

$h = 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 0 \rightarrow 1 \rightarrow 2$

Bitti: 8/10 ekleme yerleşti (2 reddedildi, tablo doluydu). Doğrusal yoklama basittir ama birincil kümelenmeye yol açar: dolu bloklar büyüdükçe yeni çakışmalar daha da büyütür.

Kod — doğrusal yoklamayla ekleme

```
bool insert(int key) {  
    int idx = key % M;  
    for (int i = 0; i < M; i++) {  
        if (state[idx] != OCCUPIED) {  
            table[idx] = key;  
            state[idx] = OCCUPIED;  
            return true;  
        }  
        idx = (idx + 1) % M;  
    }  
    return false; /* tablo dolu */  
}
```

Kod — silme bir mezar taşı bırakır

```
bool delete_key(int key) {  
    int idx = key % M;  
    for (int i = 0; i < M; i++) {  
        if (state[idx] == EMPTY)  
            return false;  
        if (table[idx] == key) {  
            state[idx] = DELETED;  
            return true;  
        }  
        idx = (idx + 1) % M;  
    }  
    return false;  
}
```

Birincil kümelenme (primary clustering)

- Çakışan anahtarlar her zaman **aynı** doğrusal yolu izler
- Dolu öbekler büyür, büyüyen öbekler daha çok çakışma çeker
- Bu kartopu etkisine **birincil kümelenme** denir
- Bölüm 9b ve 9c bunu daha az öngörülebilir bir adımla düzeltir

Karmaşıklık

- Düşük yük faktöründe ekleme/arama: $O(1)$ 'e yakın
- α 1'e yaklaştıkça: kümelenme maliyeti keskin şekilde artırır
- En kötü durum: $O(m)$ — tüm tabloyu taramak
- α 'yı 1'in belirgin altında tutmak zincirlemeden bile daha önemli

Sık yapılan hatalar

- Silinen bir hücreyi DELETED yerine EMPTY'ye temizlemek
- α 'nın 1,0'a ulaşmasına izin vermek — ekleme artık temiz bitmez
- Aramanın DELETED hücreleri de **geçmesi** gerektiğini unutmak

Mini soru

Bir anahtar silinir, bir mezar taşı bırakır.
Sonraki bir `search()` o hücreyi durmadan
geçer. Bu neden doğru davranış?

Yanıt

Çünkü silinenden sonra yerleştirilmiş bir anahtar, tam onun üzerinden yoklanmış olabilir.
Bir mezar taşında durmak, onu yanlışlıkla yok sayardı.

9b. Karesel Yoklama (Quadratic Probing)

Karesel yoklama fikri

- home 'da çakışma mı? $\text{home} + 1^2$, $\text{home} + 2^2$, ... dene
- Adım hızla büyür: 1, 4, 9, 16, ... — 1, 1, 1, ... yerine
- Çakışan anahtarları birbirinden ayırır — birincil kümelenmeyi azaltır
- Yeni tuzak: i^2 dizisi başarısız şekilde **döngüye** girebilir

Karesel yoklama, adım adım

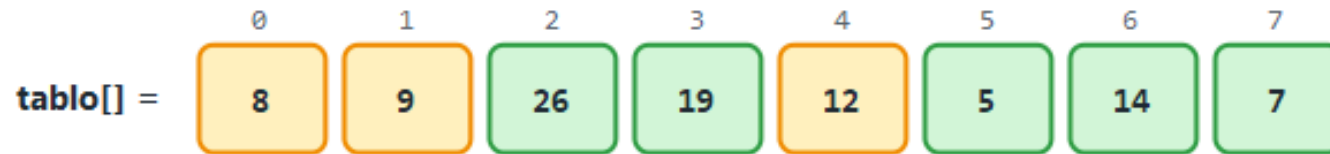
m = 13



Bitti: 10/10 ekleme yerleşti. Karesel yoklama kümelenmeyi azaltır ama m'in seçimine duyarlıdır.

Uç durum — m 2'nin kuvveti: bir döngü

$m = 8$



tablo dolu!

home=0: $0 \rightarrow 1 \rightarrow 4 \rightarrow 1 \rightarrow 0 \rightarrow 1 \rightarrow 4 \rightarrow 1$

Bitti: 8/10 ekleme yerleşti, 1 tanesi boş yer varken döngüye girdi (m asal değil / $\alpha > 0.5$). Karesel yoklama kümelenmeyi azaltır ama m 'in seçimine duyarlıdır.

Kod — karesel yoklamayla ekleme

```
bool insert(int key) {  
    int home = key % M;  
    for (int i = 0; i < M; i++) {  
        int idx = (home + i*i) % M;  
        if (state[idx] != OCCUPIED) {  
            table[idx] = key;  
            state[idx] = OCCUPIED;  
            return true;  
        }  
    }  
    return false;  
}
```

Karmaşıklık, ve m neden önemli

- Düşük yük faktöründe ekleme/arama: $O(1)$ 'e yakın
- Aynı α 'da doğrusal yoklamadan daha az kümelenme
- m asal ve $\alpha \leq 0,5$ gerektirir — yoksa i^2 dizisi boş bir hücreye ulaşmadan **döngüye** girebilir

Sık yapılan hatalar

- "Yuvarlak sayı diye" m 'yi 2'nin kuvveti seçmek
- α 'nın 0,5'i aşmasına izin vermek — döngü olası hale gelir
- Döngüye giren bir eklemeyi gerçekten dolu bir tabloyla karıştırmak

Mini soru

$m = 8$. Karesel yoklama burada neden

$m = 11$ 'e göre daha riskli?

Yanıt

8, asal değil, 2'nin kuvveti. i^2 dizisi, tabloda hâlâ boş yer olsa bile, yalnız birkaç hücre arasında döngüye girebilir.

9c. Çift Hash (Double Hashing)

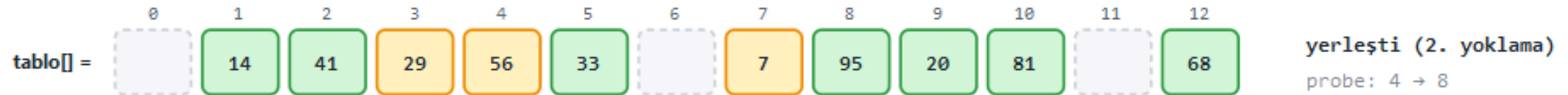
Çift hash fikri

- İki hash fonksiyonu: ev için $h1(key)$, adım için $h2(key)$
- $probe(i) = (h1(key) + i * h2(key)) \bmod m$
- Farklı anahtarlar genelde farklı adımlar alır
- Çakışan anahtarlar artık aynı yolu izlemez

Çift hash, adım adım

m = 13, R = 11

$$h1(95) = 4, h2(95) = 11 - (95 \bmod 11) = 4$$

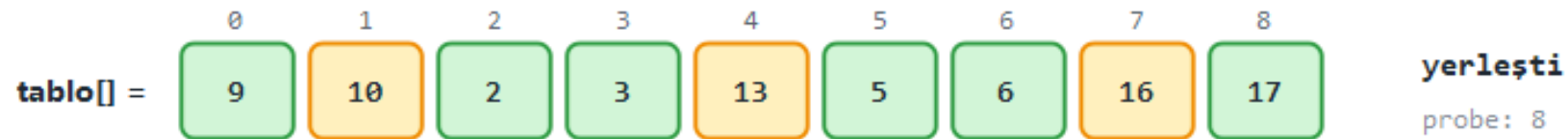


Bitti: 10/10 ekleme, toplam 15 yoklama. Aynı anahtarlarla doğrusal yoklama 10/10 yerleştirir, toplam 24 yoklamada — çift hash genelde daha az kümelenme, daha az yoklama demektir.

Uç durum — ortak çarpanlı bir döngü

m = 9, R = 7

$h1(17) = 8, h2(17) = 7 - (17 \bmod 7) = 4$



Bitti: 9/10 ekleme, toplam 18 yoklama. Aynı anahtarlarla doğrusal yoklama 9/10 yerleştirir, toplam 22 yoklamada — çift hash genelde daha az kümelenme, daha az yoklama demektir.

Kod — iki hash fonksiyonu

```
int h1(int key, int m) {  
    return key % m;  
}  
int h2(int key, int r) {  
    return r - (key % r); /* r < m, asal */  
}
```

Kod — iki hash ile yoklama

```
int idx = h1(key, M);
int step = h2(key, R);
for (int i = 0; i < M; i++) {
    if (state[idx] != OCCUPIED) {
        table[idx] = key;
        return idx;
    }
    idx = (idx + step) % M;
}
```

Karmaşıklık, ve m neden önemli

- α 1'in belirgin altındayken ekleme/arama: $O(1)$ 'e yakın
- Üç yoklama kuralının en az kümelenmesi
- Her adımın **her** hücreye ulaşması için m 'nin asal olması gerekir
- Pratikte genelde en iyi açık adresleme seçimi

Sık yapılan hatalar

- `h2(key)` 'nin **0** dönebilmesine izin vermek — sonsuz bir adım
- `m` ve `R` 'yi ortak bir çarpan paylaşacak şekilde seçmek
- `R` 'nin kendisinin asal, ve `$R < m$` olması gerektiğini unutmak

Mini soru

Doğrusal, karesel ve çift hash, iyi davranış için hepsi m 'nin asal olmasını ister.

m asal değilken hangisi **en** hoşgörülü?

Yanıt

Doğrusal yoklama. m bileşikken bile her hücreye ulaşmaya devam eder — yalnız **kümelenmesi** kötüleşir, doğruluğu değil.

10. Yeniden Hash'leme (Rehashing)

Başlangıç sorusu

Bölüm 9'daki her teknik, α 1'e yaklaştıkça yavaşlıyor. Bu gerçekleşmeden önce ne olmalı?

Yeniden hash'leme fikri

- α için bir **eşik (threshold)** seç (genelde 0,7–0,8 civarı)
- Onu aşmak bir **yeniden hash'lemeyi** tetikler: daha büyük bir tablo kur
- Yeni boyut: en az $2 * m$ olan bir sonraki **asal**
- **Her** anahtar yeniden eklenmeli — indeksi değişebilir

Neden her anahtar taşınmalı

- $h(k) = k \bmod m$ — m 'yi değiştirin, $h(k)$ genelde değişir
- Eski hücreleri doğrudan yenilerine kopyalamak yanlış olurdu
- Bunun yerine: her anahtarı yeni, boş tabloya **yeniden ekle**
- Bu, zaten yazılmış olan aynı `insert()` 'ü kullanır

Yeniden hash'leme, adım adım

eşik (threshold) = 0.8

$$\alpha = 10/13 = 0.77$$

insert(40) → idx 10



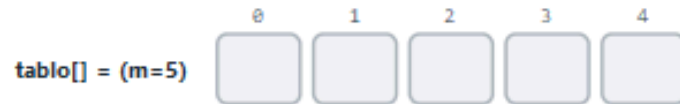
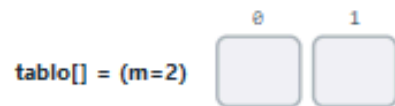
Bitti: 10 anahtar eklendi, tablo $m_0 = 6$ 'dan $m = 13$ 'e büyüdü. Yeniden hash'leme pahalıdır ($O(n)$) ama seyrek — amortize edilmiş ekleme yine $O(1)$ kalır.

Uç durum — art arda iki kez büyümek

eşik (threshold) = 0.8

$\alpha = 6/11 = 0.55$

insert(16) → idx 6



Bitti: 6 anahtar eklendi, tablo $m_0 = 2$ 'dan $m = 11$ 'e büyüdü. Yeniden hash'leme pahalıdır ($O(n)$) ama seyrek — amortize edilmiş ekleme yine $O(1)$ kalır.

Kod — bir sonraki asal boyutu bulmak

```
int is_prime(int x) {  
    if (x < 2) return 0;  
    for (int i=2; i*i<=x; i++)  
        if (x % i == 0) return 0;  
    return 1;  
}  
int next_prime(int x) {  
    while (!is_prime(x)) x++;  
    return x;  
}
```

Kod — büyötmek ve yeniden eklemek

```
void rehash(void) {  
    int newM = next_prime(2 * m);  
    /* ... newState, newTable ayır ... */  
    for (int i = 0; i < m; i++)  
        if (state[i] == OCCUPIED)  
            insert_into(newState, newTable,  
                        newM, table[i]);  
    m = newM; /* ... yeni dizileri deęiřtir ... */  
}
```

Amortize karmaşıklık

- Tek bir yeniden hash'leme $O(n)$ 'e mal olur — her anahtar yeniden eklenir
- Yeniden hash'lemeler **nadiren** olur: tablo boyutu en az iki katına çıkar
- Birçok eklemeye yayıldığında, ortalama maliyet $O(1)$ kalır
- Bu, Hafta 2'nin dinamik dizisiyle aynı **amortize** fikri

Sık yapılan hatalar

- Hiç yeniden hash'lememek — α sınırsız büyür
- Asal olmayan bir boyuta büyümek
- Bir eşiği geçmek yerine her tek eklemede yeniden hash'lemek

Mini soru

$m = 6$ olan bir tabloda $n = 5$ anahtar var,
eşik $0,8$. Bir anahtar daha ekleniyor. Bir
yeniden hash'leme tetiklenir mi, hangi boyuta?

Yanıt

Evet. $\alpha = 6/6 = 1,0 > 0,8$ — bir yeniden hash'leme tetiklenir. Yeni boyut: `next_prime(12) = 13`.

11. Bir Çakışma-Çözme Tekniği Seçmek

Zincirleme mi, açık adresleme mi

	Zincirleme	Açık adresleme
Ekstra bellek	Evet, düğüm başına	Hayır, yerinde
Yüksek α	Zarifçe bozulur	Keskin bozulur
Silme	Basit	Mezar taşı ister

Açık adresleme içinde seçim

- Basit kod, α düşük tutuluyor mu? **Doğrusal** yoklama yeter
- Daha az kümelenme isteniyor, asal m kabul ediliyor mu?
Karesel yoklama
- Mevcut en iyi dağılım isteniyor mu? **Çift** hash

Pratik bir kural

- Silmeler sık, ya da bellek sıkı değil mi? **Zincirleme**
- Bellek önemli, α 1'in belirgin altında mı? **Açık adresleme**
- Her iki durumda: α çok büyümeden **yeniden hash'le**
- Evrensel olarak "en iyi" seçim yok — yalnız **sizin durumunuz** için en iyisi

Mini soru

Bir hash tablosu milyonlarca anahtar tutacak,
silme neredeyse hiç yok, ve bellek sıkı.
Hangi aile en uygun, neden?

Yanıt

Açık adresleme, muhtemelen **çift hash**.
Endişelenecek silme yok, milyonlarca zincir bağlantısı için ekstra düğüm belleği de yok.

Özet — sıralı veride daha hızlı arama

Fikir	Anahtar gerçek
Sıçramalı arama	$O(\sqrt{n})$, block = $\sqrt{n}$
Enterpolasyon araması	Düzgün veride $O(\log \log n)$
Üstel / Fibonacci	Sınırsız n / bölme yok

Özet — hash'leme ve çakışmalar

Fikir	Anahtar gerçek
Hash fonksiyonu	$h(k) = k \bmod m$, $O(1)$, m asal
Zincirleme	Kova başına bağlı liste, $O(1 + \alpha)$
Açık adresleme	Yerinde, mezar taşı ister, yeniden hash

Büyük resim

Sıralı veri, ekstra yapıdan yararlanarak ikili aramadan hızlı aramayı destekler. Hash'leme karşılaştırmayı tamamen bırakır: bir konum hesapla, çakışmaları ele al, tablo dolmadan büyüt.

Öz değerlendirme turu

Dört kısa soru. Yanıt bir sonraki slaytta görünmeden önce düşünün. Tam alıştırımlar ve on soruluk bir sınav hafta notlarında.

1. Bir dizide 400 eleman var. Sıçramalı arama hangi blok boyutunu kullanır, ve en kötü durumda yaklaşık kaç karşılaştırma?

$\text{block} = \text{sqrt}(400) = 20$. En kötü durum: yaklaşık $20 + 20 = 40$ karşılaştırma — doğrusal aramanın 400'ünden çok az.

2. Enterpolasyon aramasının formülü neden `arr[hi] == arr[lo]` 'ya karşı bir koruma ister?

Koruma olmadan, formülün paydası sıfır olurdu — tümü eşit bir aralık $\text{arr}[\text{hi}] - \text{arr}[\text{lo}] = 0$ yaptığı için, bu sıfıra bölme hatası demek.

3. Bir hash tablosunun boyutu m neden genelde asal seçilmeli?

Asal bir m , çoğu anahtar örüntüsüyle ortak bir çarpan paylaşmaz, bileşik bir m 'nin (10'un kuvveti gibi) yol açabileceği ciddi kümelenmeyi önler.

4. Açık adreslemede silinen bir hücre neden sade boş bir hücre değil, bir mezar taşı olmalı?

Sonraki bir arama, gerçekten boş olan ilk hücrede durur. Boşa temizlemek, ondan sonra yerleştirilmiş anahtarların yoklama zincirini yanlışıyla keserdi.

Gelecek hafta

Hafta 7 — Proje Sunumları

Bu hafta yeni algoritma yok: takımlar projelerinin arama ya da hash'leme bileşenini sunar, ardından Hafta 8'in sınavı, Hafta 9'da graf algoritmaları sürüyor.

Kaynaklar (1/2)

- Ders izlencesi, Hafta 6: `docs/syllabus/syllabus.en.md`
- Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 4. baskı. MIT Press
- Knuth. *The Art of Computer Programming, Cilt 3: Sorting and Searching*, 2. baskı
- Bentley, J., Yao, A. (1976). "An almost optimal algorithm for unbounded searching"
- Luhn, H. P. (1953). IBM iç yazısı, hash'leme üzerine

Kaynaklar (2/2)

- Kiefer, J. (1953). "Sequential minimax search for a maximum"
- Peterson, W. W. (1957). "Addressing for random-access storage"
- Sedgewick, Wayne. *Algorithms*, 4. baskı. Addison-Wesley
- [williamfiset/Algorithms](#) · Programiz DSA